Workload-Aware Adaptive Page Replacement

Name:Arpit Raj

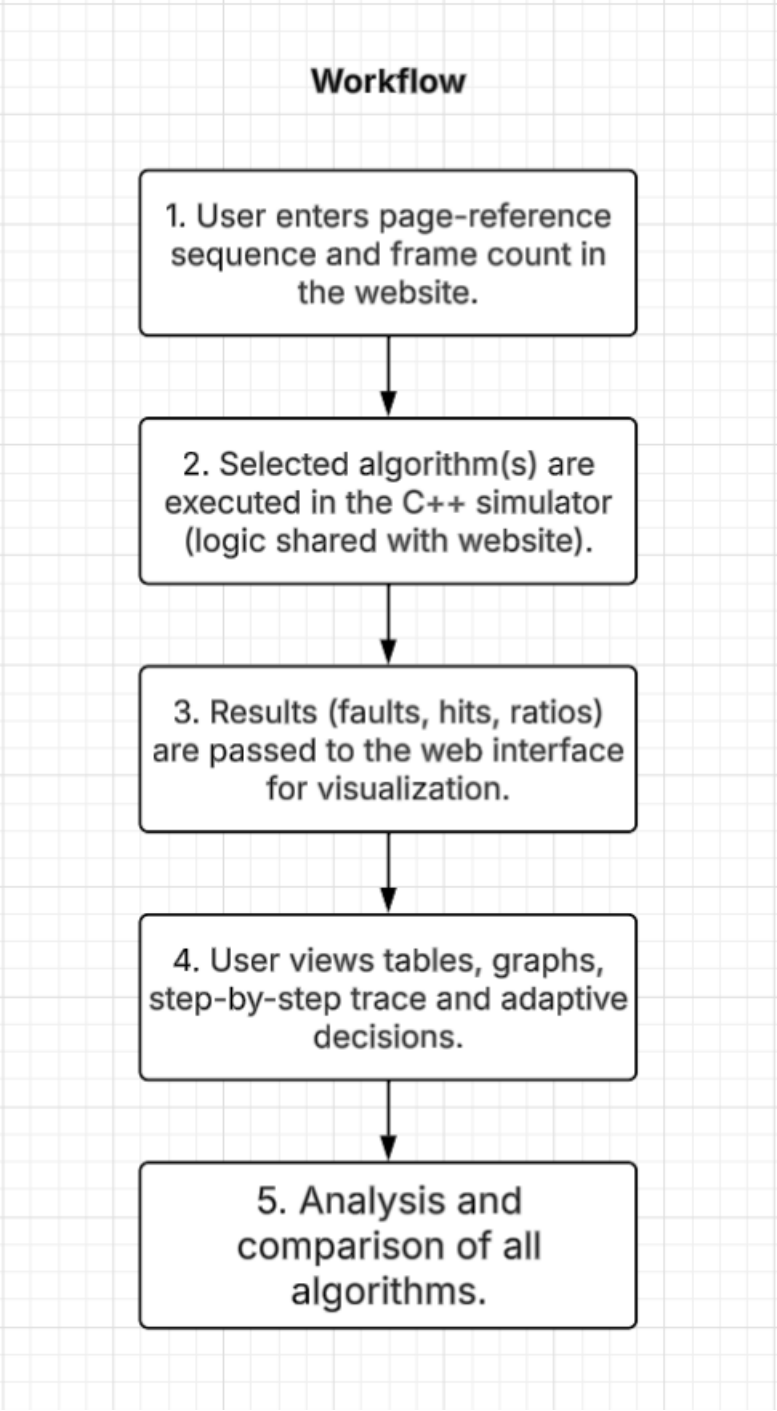
Reg.No:24BCE2897

1. Project Overview

Workload-Aware Adaptive Page Replacement is an Operating Systems project that demonstrates how page replacement algorithms manage memory when the number of pages requested by a process exceeds the available physical memory frames. When a required page is not present in memory, a page fault occurs, and the operating system must decide which existing page to replace.

This project implements and compares four page replacement algorithms: First-In, First-Out (FIFO), Least Recently Used (LRU), Optimal, and Adaptive Page Replacement. It includes a C++ simulator and a standalone web-based simulator that displays algorithm performance through tables, graphs, and step-by-step execution traces.

2. Workflow



3. Implementation Details

The project is implemented using two components: a C++ command-line simulator and a web-based visualization interface.

C++ Simulator: The C++ program implements FIFO, LRU, Optimal, and Adaptive page replacement algorithms. It accepts a page-reference sequence and the

number of available frames, simulates page accesses, and calculates page faults, page hits, hit ratio, and fault ratio.

Web Interface: The website is developed using HTML, CSS, and JavaScript. HTML defines the structure, CSS provides the layout and styling, and JavaScript implements the simulation logic, calculates performance metrics, and displays the results through tables, graphs, and step-by-step execution traces.

Adaptive Algorithm: The Adaptive approach maintains shadow simulations for FIFO and LRU and compares their page-fault counts over a recent window of up to eight references, including the current reference. It selects the policy with fewer faults and chooses FIFO when both have equal fault counts. The selected policy determines which page is replaced when necessary.

Input and Output: The user can provide a custom page-reference sequence or select a supported workload pattern and specify the number of frames. The program then displays the performance metrics and replacement behavior for comparison.

Execution: The C++ simulator is compiled using a compatible C++ compiler and executed through the terminal in VS Code. The web interface runs locally in a web browser by opening the index.html file.

The two components provide complementary ways to execute, analyze, and demonstrate page replacement algorithms.

4. Technology Stack and Configuration

The project uses the following technologies:

- C++: Used to implement the command-line page replacement simulator.
- HTML: Defines the structure and components of the website.
- CSS: Provides the website layout, dark theme, colors, tables, and responsive styling.
- JavaScript: Implements the website simulations, calculates performance metrics, and updates the results and graphs.
- Web browser: Used to run the website locally.
- GitHub: Used to store and share the project source code and documentation.

The website consists of three files: index.html, style.css, and script.js. It can be run locally by opening index.html in a web browser. The website does not require a database, login system, or additional framework.

The C++ simulator requires a compatible C++ compiler. The exact compiler version and compilation command should be documented according to the development environment used.

5. Algorithms Implemented

FIFO (First-In, First-Out): Replaces the page that has been in memory for the longest time. It is simple to implement but may replace a page that will soon be needed again.

LRU (Least Recently Used): Replaces the page that has not been accessed for the longest time. It uses recent access history to make replacement decisions.

Optimal Page Replacement: Replaces the page whose next use is farthest in the future. It is primarily used as a theoretical benchmark because it requires knowledge of future page references.

Adaptive Page Replacement: The website uses two hidden shadow memories to track FIFO and LRU performance over a recent window of up to eight references, including the current reference. When a replacement is needed, it selects the strategy with fewer faults in that window. If both strategies have equal faults, FIFO is selected. This approach uses observed references rather than future information. Adaptive is a heuristic and is not guaranteed to outperform FIFO or LRU on every workload.

The C++ and website implementations should be documented separately if their Adaptive decision logic differs.

6. Website Features and Implementation

The website provides a simple dashboard for comparing all four algorithms using the same page-reference sequence and number of frames.

Its main features include:

- Input fields for the page-reference sequence and number of frames.
- A simulation button to execute the algorithms.
- A results table displaying page faults, page hits, hit ratio, and fault ratio.

- A bar chart comparing page faults across algorithms.
- A step-by-step simulation table showing frame contents and whether each reference results in a hit or fault.
- An Adaptive decision table showing the selected strategy and replacement information.

These features make it easier to observe how different replacement strategies behave under different workloads.

7. Testing and Validation

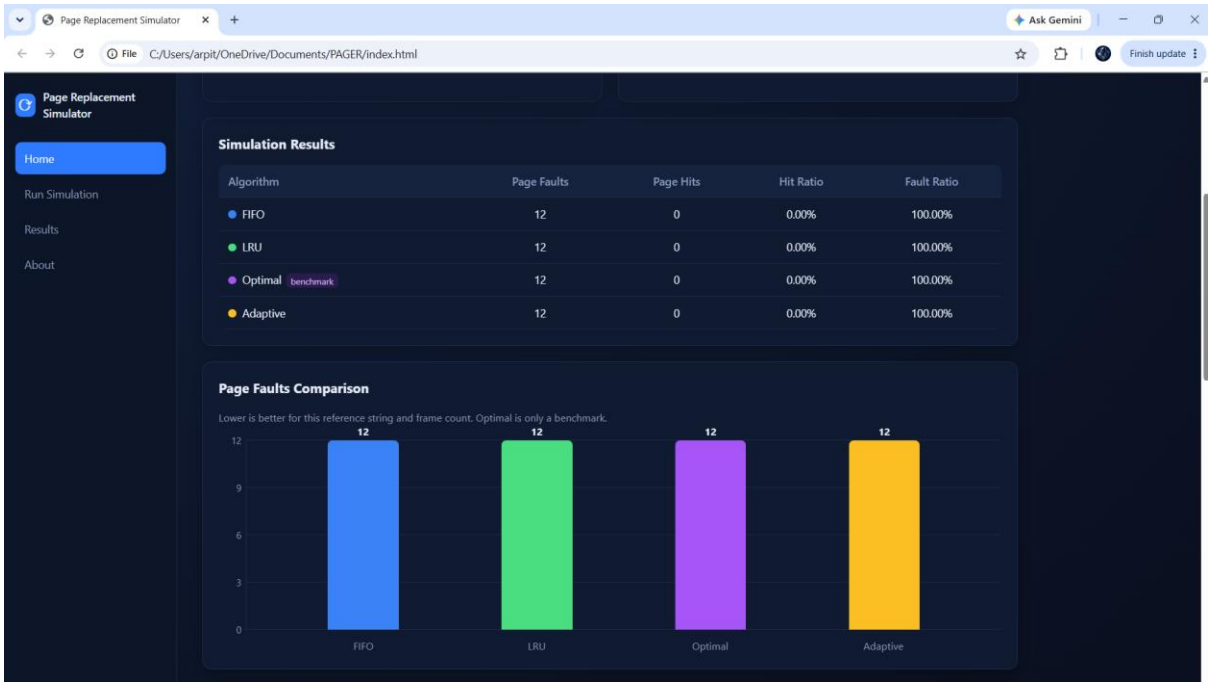
The simulator should be tested with sequential, repeating, textbook, and mixed page-reference sequences. Tests should use different numbers of memory frames to observe changes in algorithm performance.

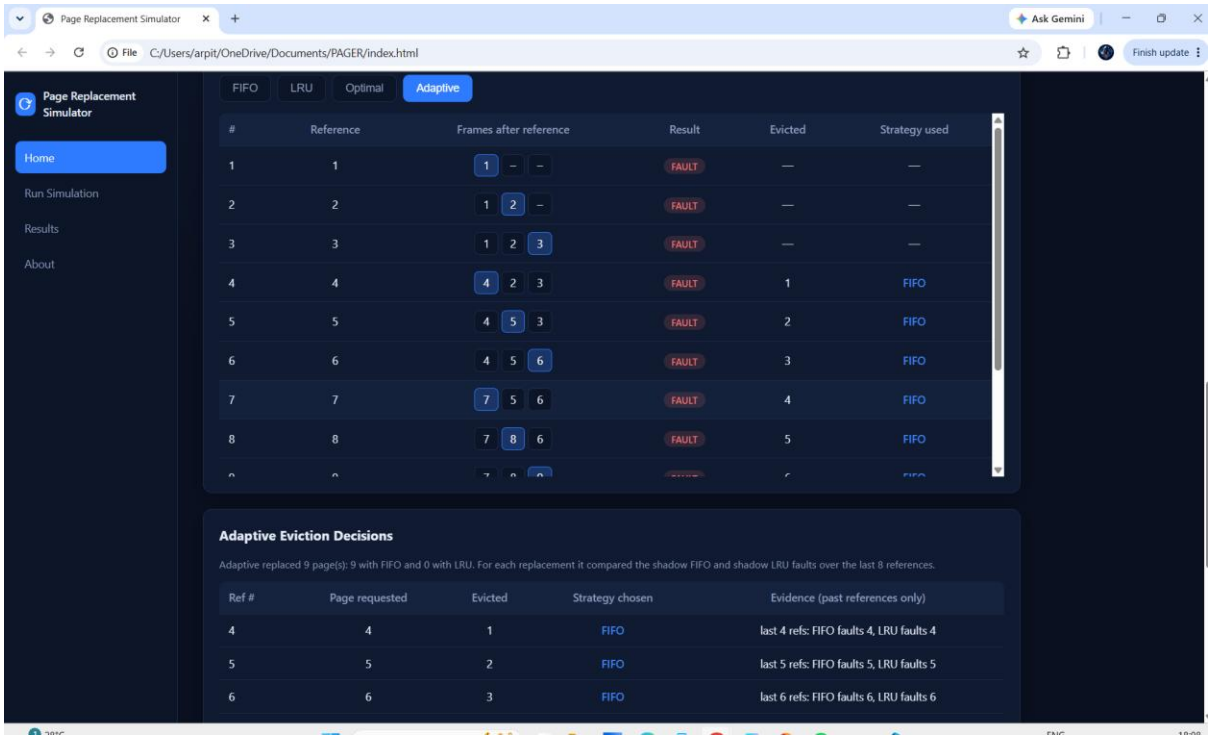
The following checks should be performed:

1. The number of page hits plus page faults must equal the total number of page references.
2. The hit ratio and fault ratio must add up to 1, or 100% when expressed as percentages.
3. The results table and bar chart must update when new inputs are entered.
4. The step-by-step trace must show the correct frame contents and hit or fault status.
5. Adaptive decisions must be displayed when replacements occur.
6. Optimal should produce no more page faults than another correctly implemented algorithm for the same reference sequence and frame count.

8. Result and Screenshots

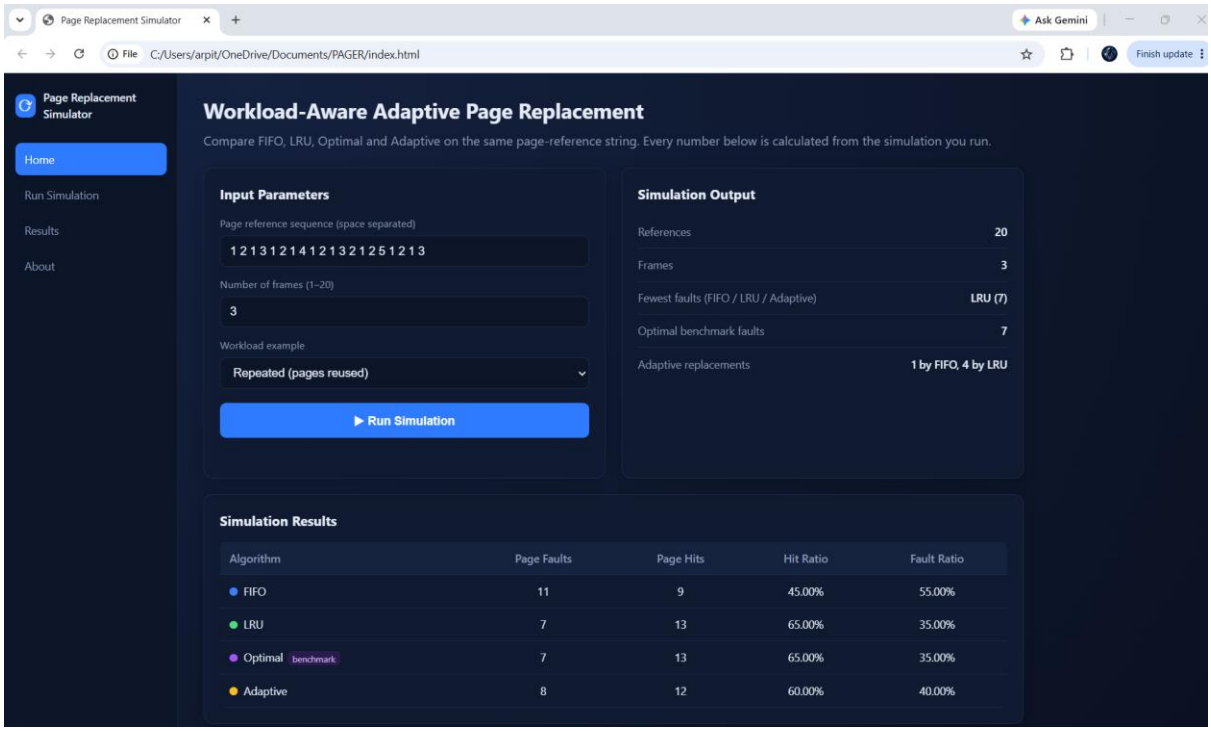
Sequential Reference:

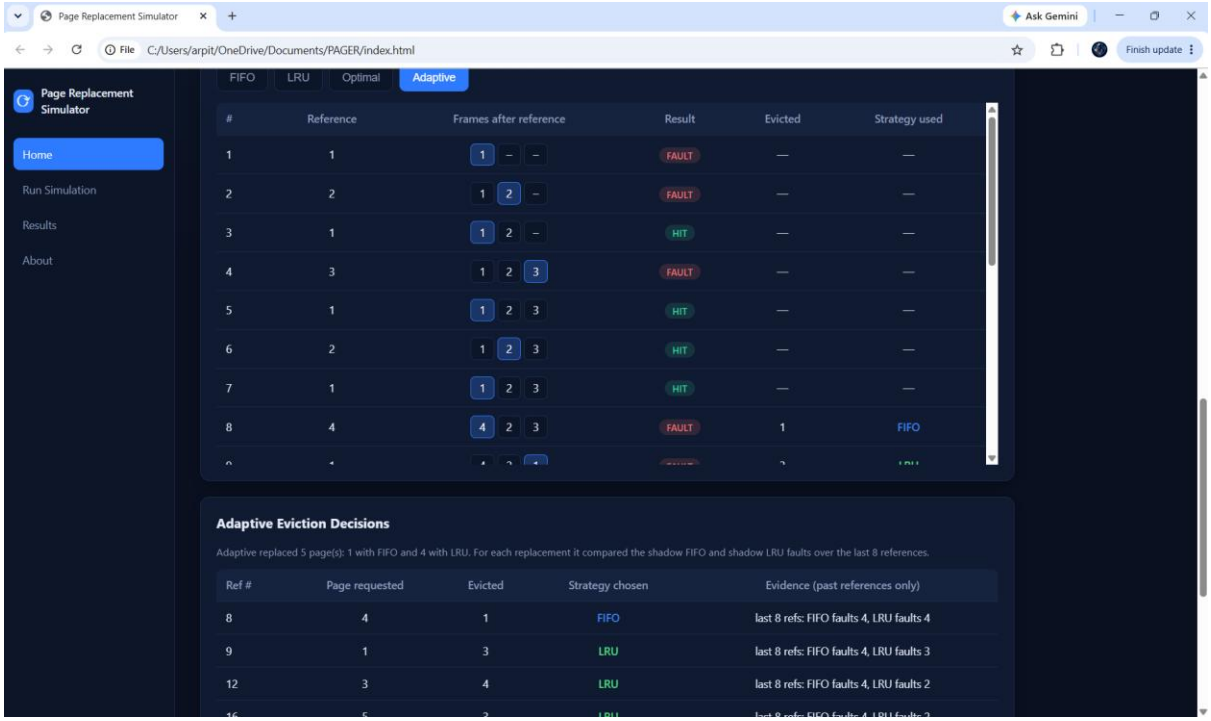




Sequential Workload — Simulation results showing the performance of FIFO, LRU, Optimal, and Adaptive algorithms for a sequential page-reference pattern.

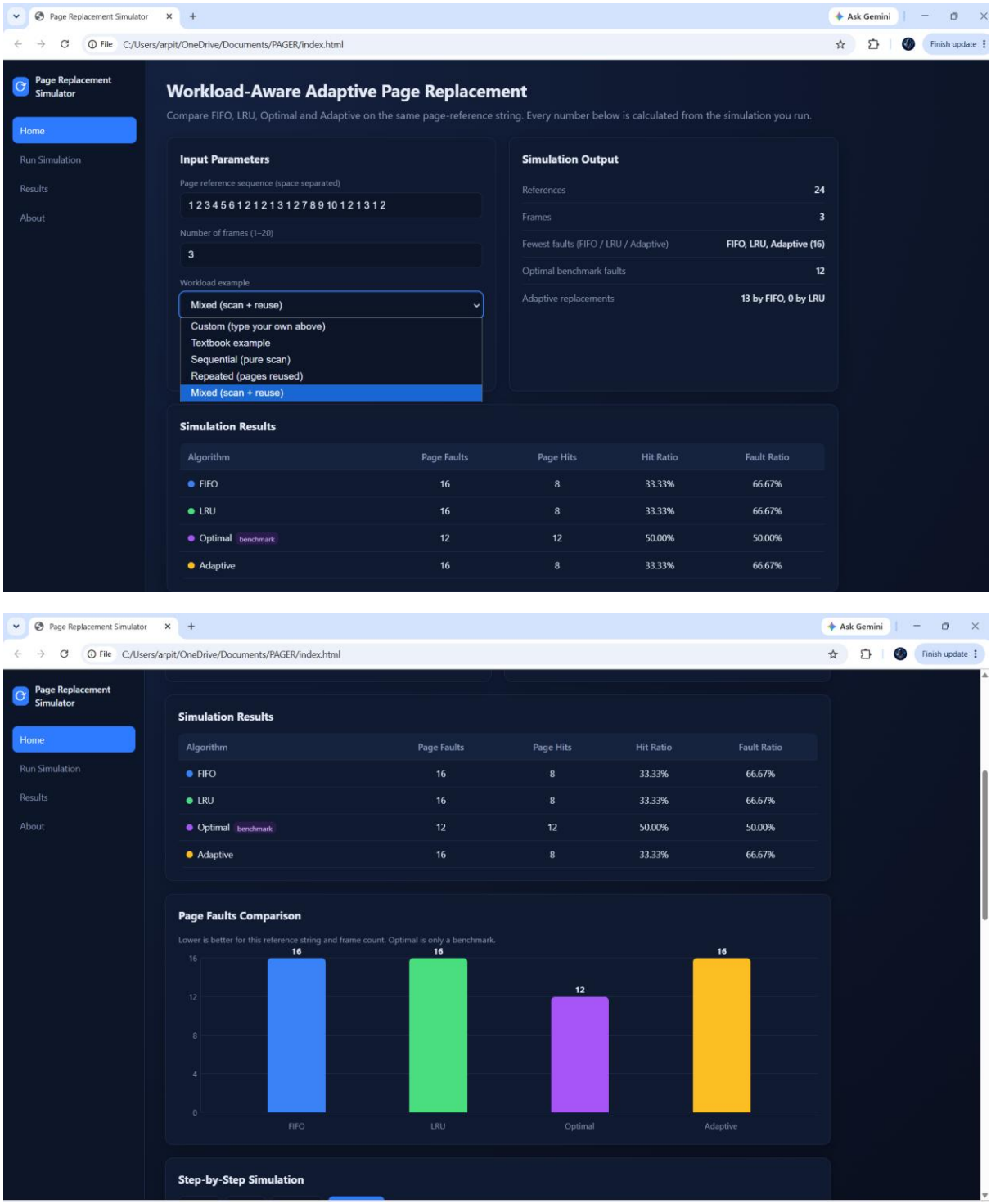
Repeated Reference:





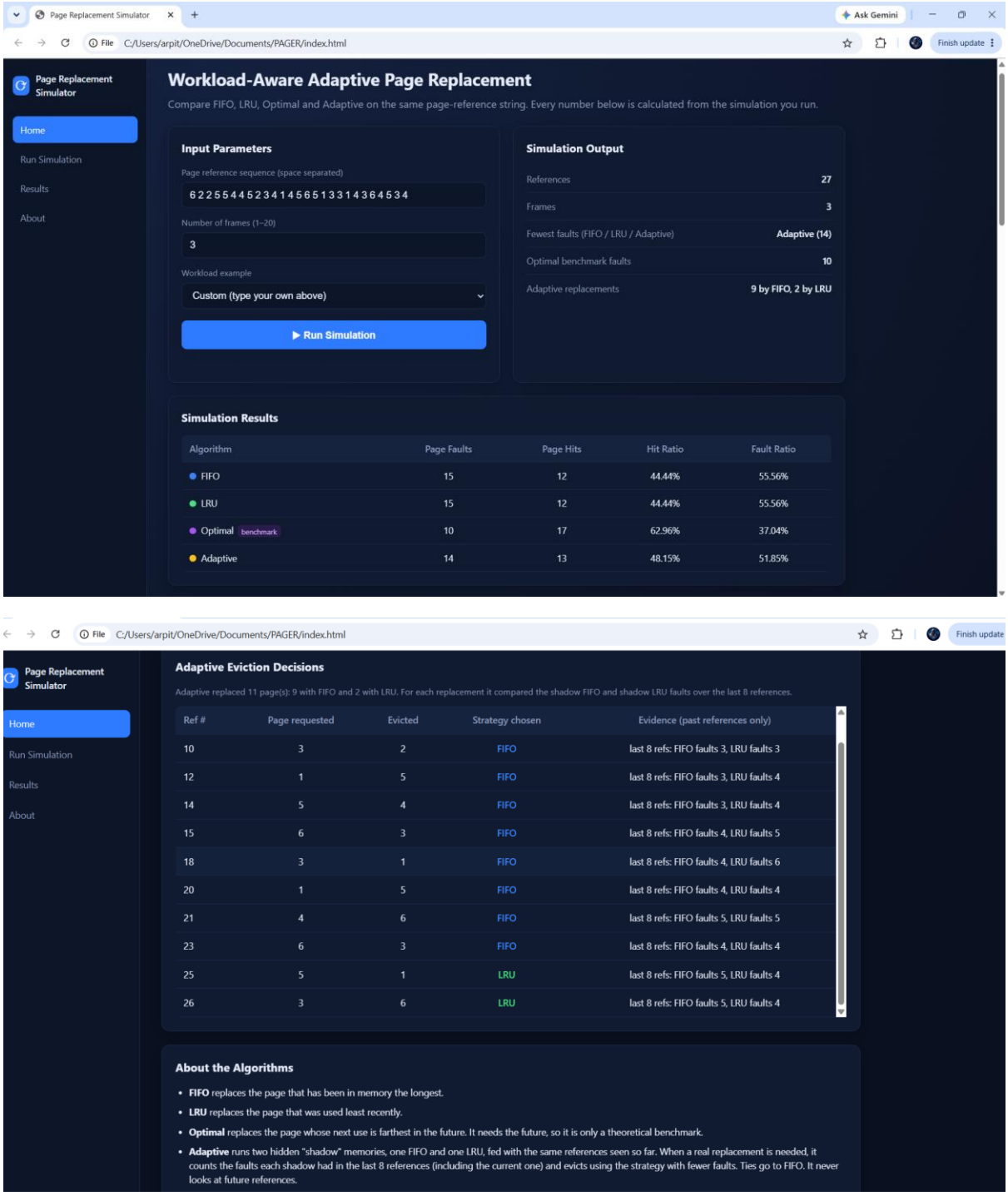
Repeated Workload — Simulation results demonstrating algorithm performance when the same pages are accessed repeatedly.

Mixed Reference:



Mixed Workload — Comparison of page faults, page hits, and hit and fault ratios for a mixed page-reference pattern.

Custom Reference:



Custom Workload — Results for a user-defined page-reference sequence and frame count, including the Adaptive algorithm's replacement decisions where displayed.

9. GitHub Repository and Submission

GitHub Repository Link: [https://github.com/arpitraj702-del/Workload-Aware-Adaptive-Page-Replacement.git]

10. Results and Discussion

The project was evaluated using sequential, repeated, mixed, and custom page-reference workloads. The four algorithms were compared using page faults, page hits, hit ratio, and fault ratio.

The results show that the performance of a page replacement algorithm depends on the reference sequence and the number of available memory frames. Sequential workloads can introduce many new pages, while repeated workloads may benefit from keeping frequently accessed pages in memory. Mixed and custom workloads help demonstrate how the algorithms behave under different access patterns.

Optimal serves as a theoretical benchmark because it uses knowledge of future page references to minimize page faults for a given sequence and frame count. FIFO uses the order in which pages enter memory, whereas LRU uses recent access history.

The Adaptive algorithm compares the recent page-fault performance of FIFO and LRU using its observation window. It selects the policy with fewer observed faults, with FIFO chosen in the event of a tie. This allows the algorithm to respond to recent workload behavior; however, it does not guarantee fewer page faults than both FIFO and LRU for every input.

The screenshots provide evidence of the simulations and the displayed performance metrics. Overall, the project demonstrates how workload characteristics influence page replacement performance and how an adaptive strategy can make replacement decisions based on observed results.

11. Conclusion

The Workload-Aware Adaptive Page Replacement project demonstrates the operation of common page replacement algorithms and compares their performance using page faults, page hits, and related metrics. The web interface provides a visual and interactive way to understand memory

replacement decisions. The Adaptive strategy illustrates how recent workload behaviour can guide the selection between FIFO and LRU, while Optimal serves as a theoretical benchmark for comparison.

The project combines operating system concepts with C++ programming and web technologies to provide an educational tool for understanding virtual memory management.